로봇러닝 인턴십 1주차 세미나

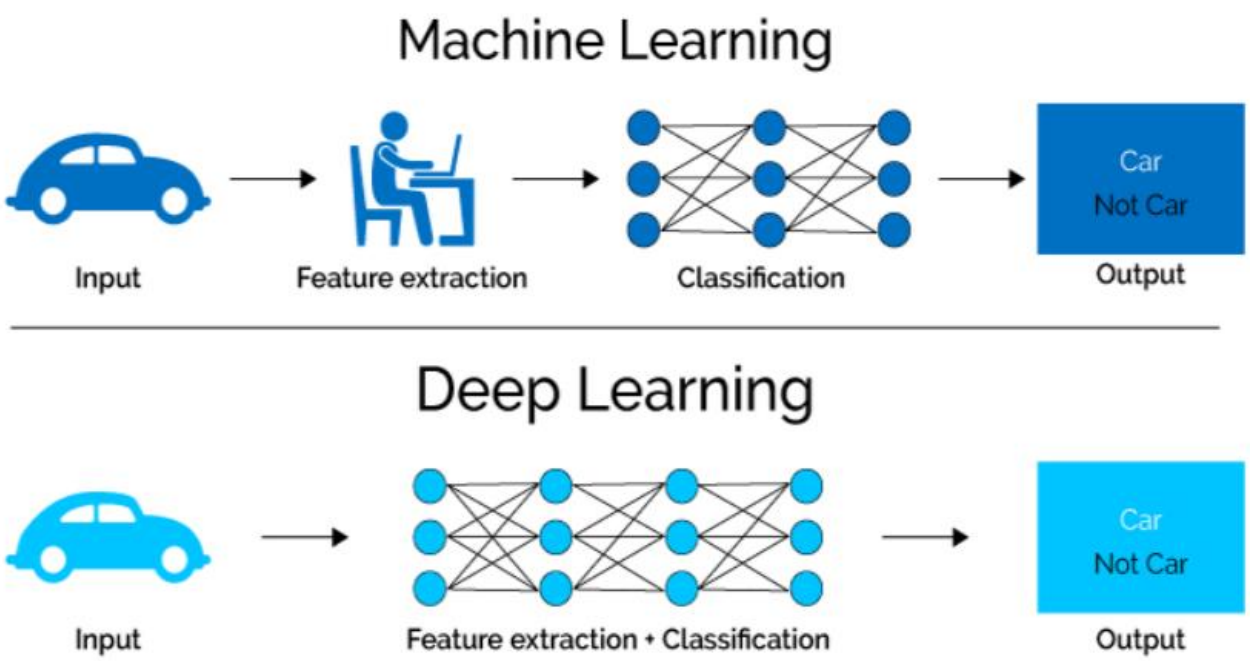
딥러닝 기초 학습

PRESENTER 이용신

DATE 2026. 01. 08

머신러닝 vs 딥러닝

특징 추출(Feature Extraction)의 주체에 따른 핵심적 차이



Machine Learning

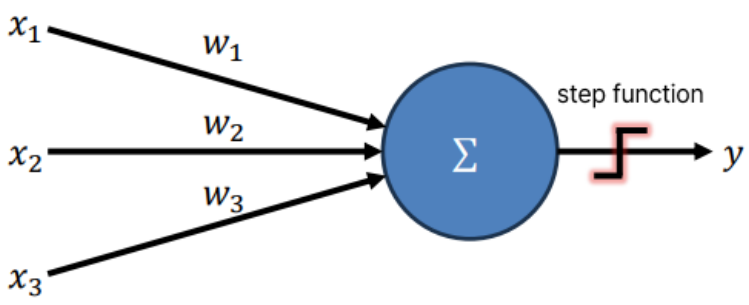
- **수동 특징 추출 (Manual):** 사람이 직접 유용한 특징을 선별하여 컴퓨터에 입
- **주요 알고리즘:** SVM, Random Forest, Decision Tree 등
- **데이터/연산량:** 상대적으로 적은 데이터로도 학습 가능, 낮은 연산 비용

Deep Learning

- **자동 특징 학습 (Automated):** 신경망이 데이터 패턴을 스스로 파악하여 특징 추출
- **주요 모델:** CNN, RNN, Transformer (End-to-End Learning)
- **데이터/연산량:** 대규모 데이터와 고성능 GPU 연산 필수

퍼셉트론 (Perceptron)

단순한 선형 분류기(Linear Classifier)



$$y = s\left(\sum_{i=0}^{N} w_i x_i\right)$$

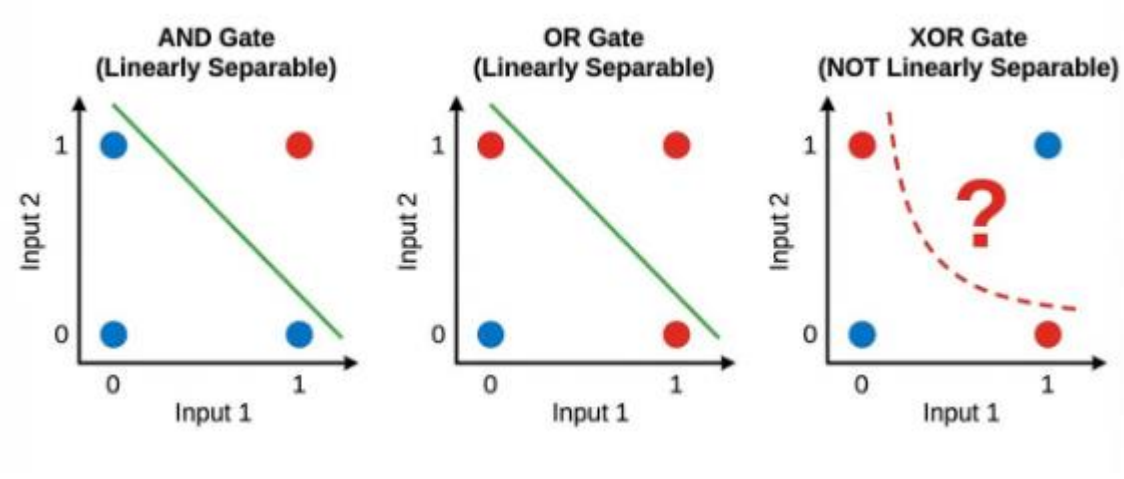
$$s(v) = \begin{cases} 1 & \text{if } v > \theta \\ 0 & \text{otherwise} \end{cases}$$

기본 구조와 동작 원리

입력 신호(x)에 가중치(w)를 곱하고 바이어스(b)를 더한 값이 임계값을 넘으면 활성화되는 구조

선형 논리 게이트 구현

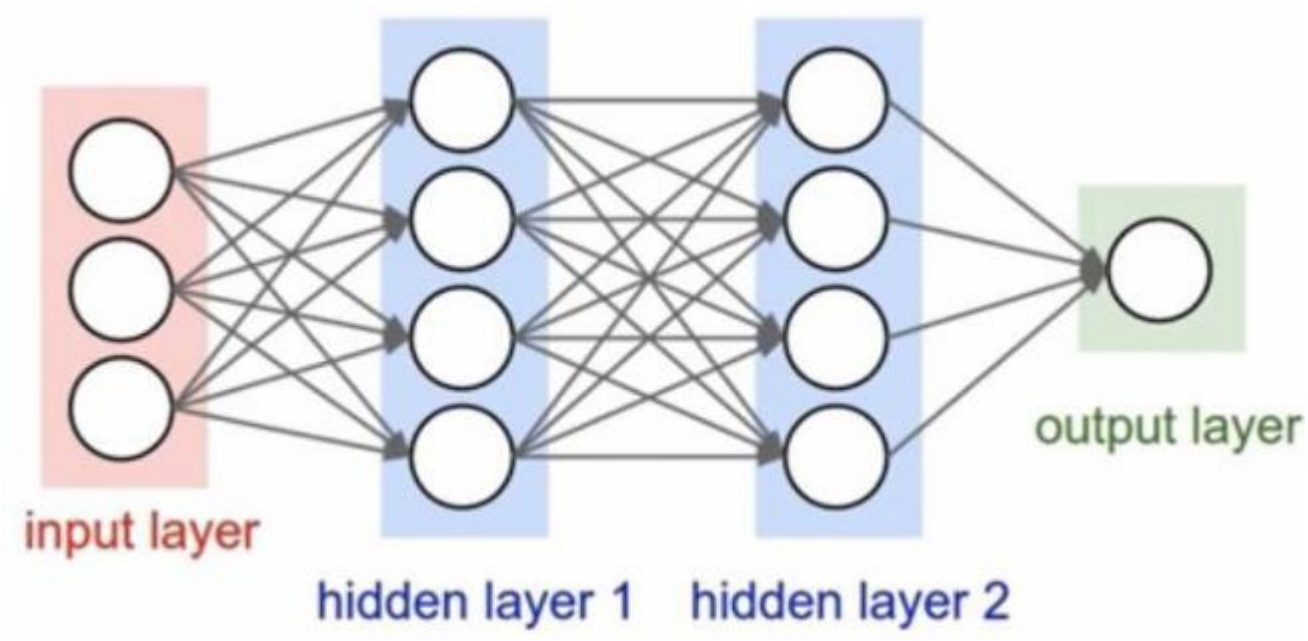
AND, OR 연산과 같이 하나의 직선으로 데이터 영역을 명확히 나눌 수 있는 문제는 완벽하게 학습 가능



치명적 한계: XOR 문제

단일 퍼셉트론은 선형 분류기(Linear Classifier)이기 때문에, 비선형 패턴인 **XOR(배타적 논리합)** 문제를 해결할 수 없음

다층 퍼셉트론
Multi-LayerPerceptron (MLP)



다층 퍼셉트론

층이 두개 이상 존재하는 신경망

은닉층

입력층과 출력층을 제외한 층(layer)

활성화 함수(Activation function)

비선형성을 부여하는 함수

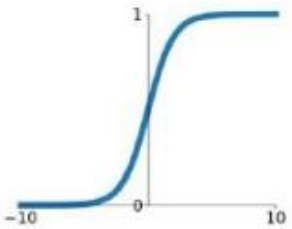
순전파와 역전파

활성화 함수 (Activation Function)

신경망에 **비선형성(Non-linearity)**을 부여하여 복잡한 데이터 패턴 학습을 가능하게 하는 핵심 요소

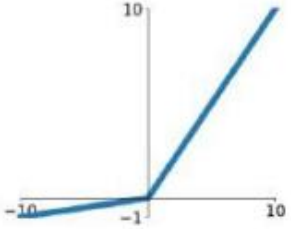
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



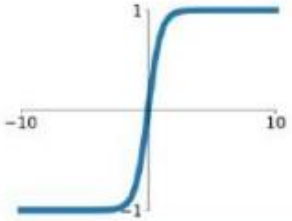
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

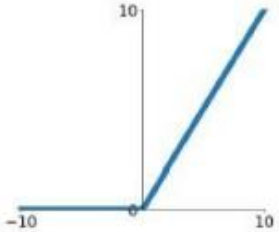


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

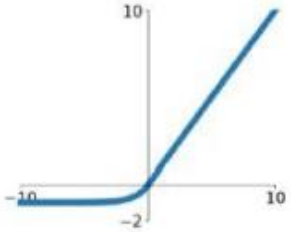
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Forward & Backward Pass

예측과 가중치 업데이트

1. Forward Pass (순전파)

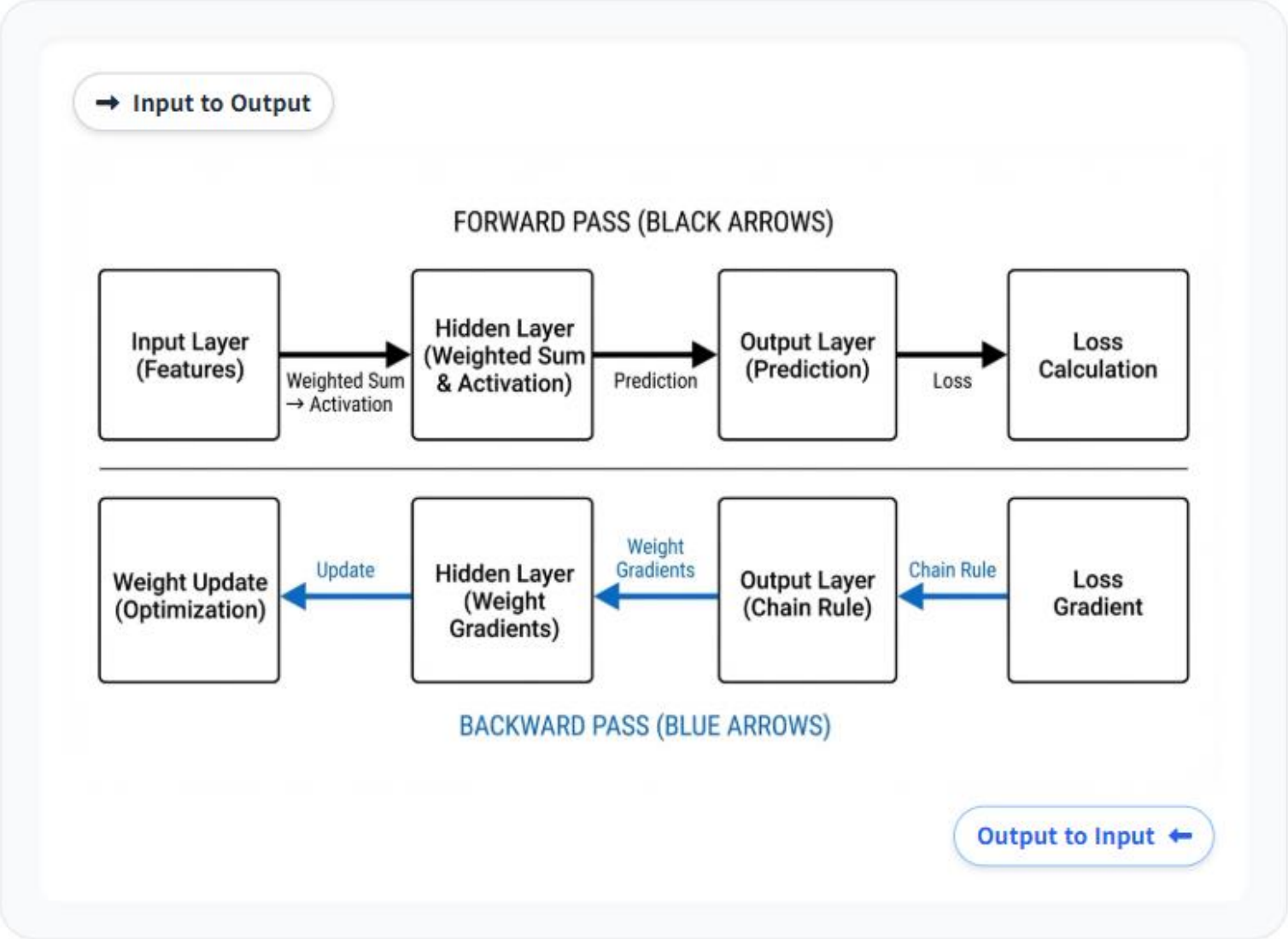
Prediction Phase

입력 데이터를 가중치(Weight)와 함께 연산하여 출력층으로 전달합니다. 이 과정에서 모델의 **예측값**과 실제 정답 사이의 **손실(Loss)**을 계산합니다.

2. Backward Pass (역전파)

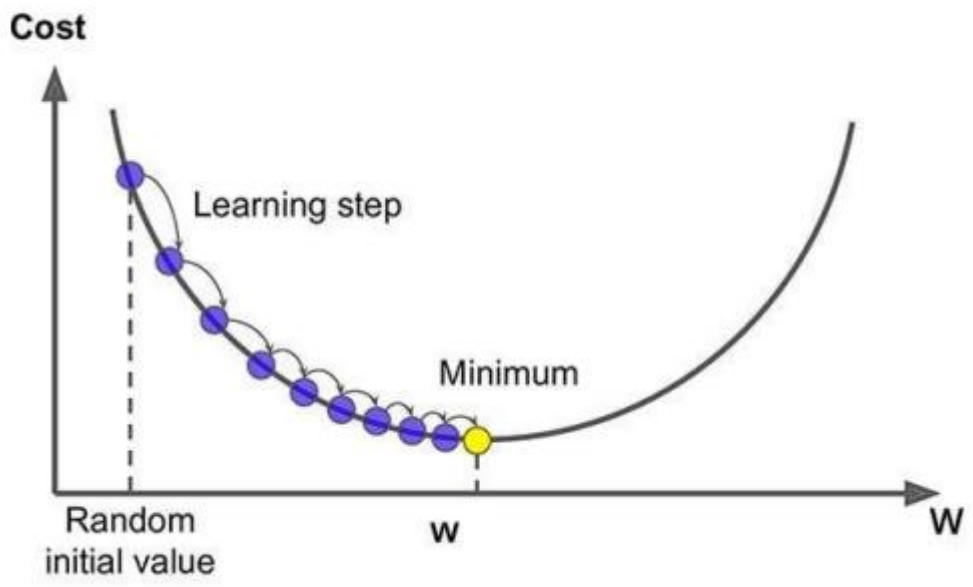
Learning Phase

계산된 손실 정보를 바탕으로, 출력층에서 입력층 방향으로 **기울기(Gradient)**를 역추적합니다. 이를 통해 각 가중치를 얼마나 수정해야 할지 결정합니다.



Optimization (최적화)

손실(Loss)을 최소화하여 최적의 가중치를 찾아가는 여정



$$W := W - \alpha \frac{\partial}{\partial W} \mathrm{cost}(W)$$

목표: Loss 최소화

모델의 예측값과 실제값 사이의 오차인 손실 함수의 값을 가장 작게 만드는 가중치(Weight)를 탐색

Gradient Descent (경사 하강법)

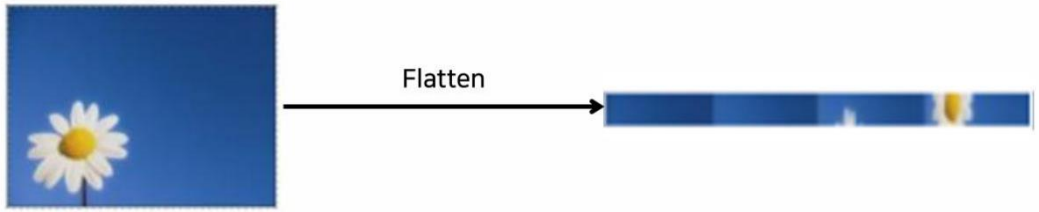
현재 위치에서 기울기(Gradient)가 가리키는 방향, 즉 에러가 줄어드는 방향으로 파라미터를 조금씩 조정

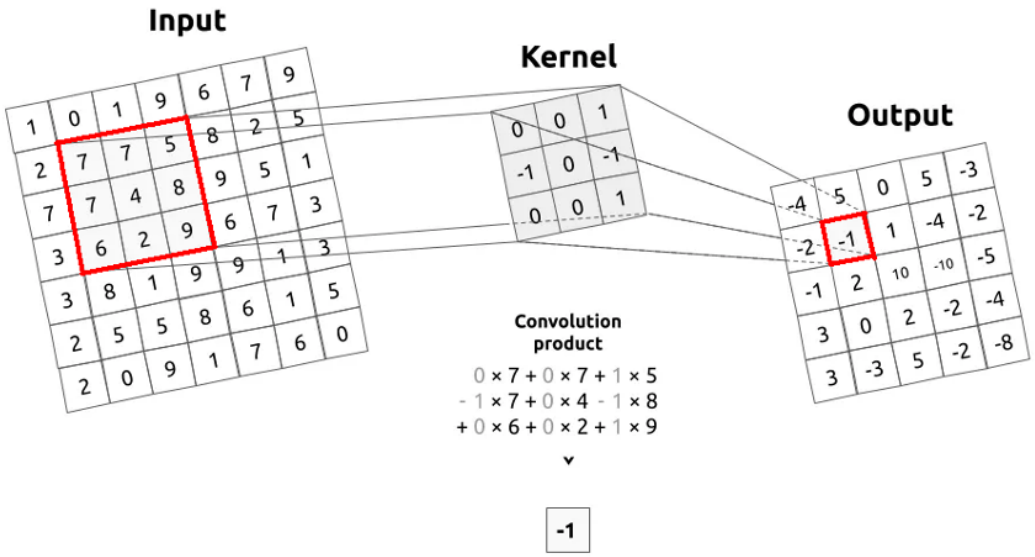
최적의 경로 탐색

학습률(Learning Rate)을 조절하여 Local Minima나 Saddle Point에 빠지지 않음

CNN: Convolution Neural Network

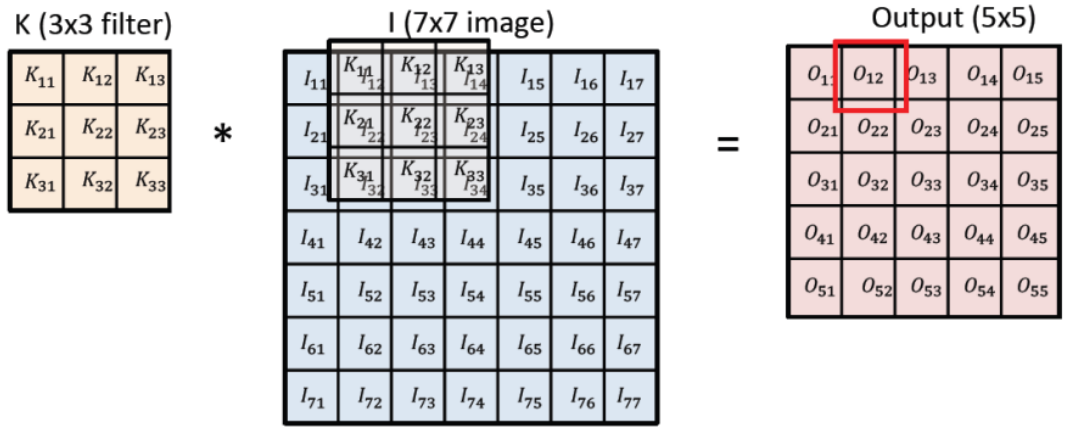
이미지의 2차원 정보를 학습





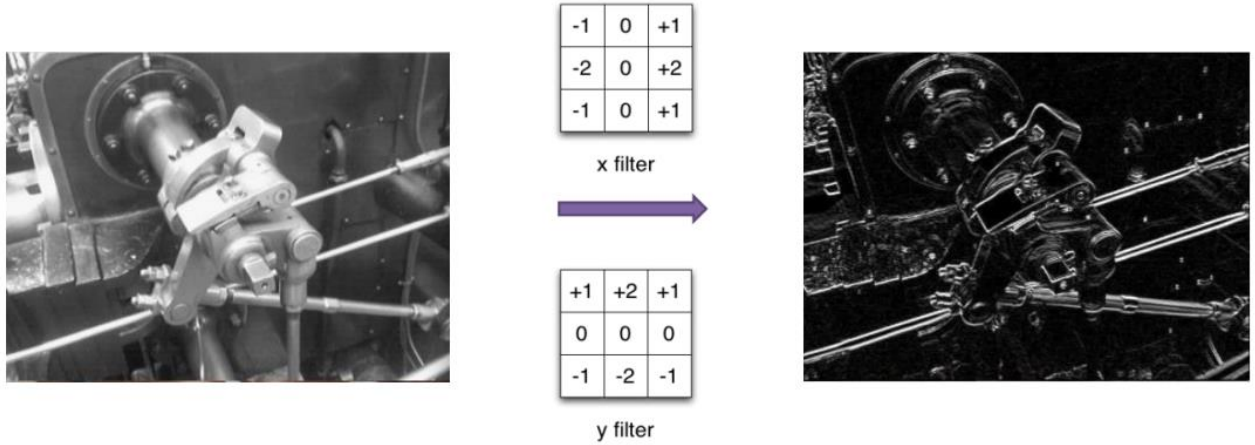
합성곱
(Convolution)

특정크기의 필터(Kernel)를 일정
간격으로 이동시키며 연산을 수행



$$O_{11} = I_{11}K_{11}+I_{12}K_{12}+I_{13}K_{13}+I_{21}K_{21}+I_{22}K_{22}+I_{23}K_{23}+I_{31}K_{31}+I_{32}K_{32}+I_{33}K_{33}+bias$$

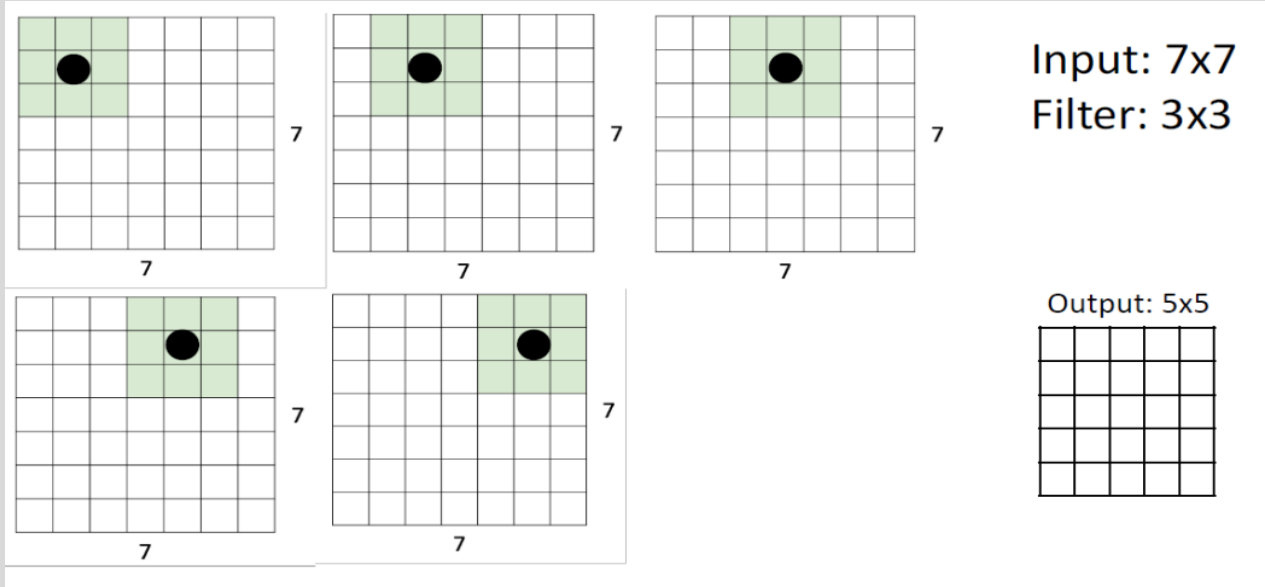
$$O_{12} = I_{12}K_{11}+I_{13}K_{12}+I_{14}K_{13}+I_{22}K_{21}+I_{23}K_{22}+I_{24}K_{23}+I_{32}K_{31}+I_{33}K_{32}+I_{34}K_{33}+bias$$

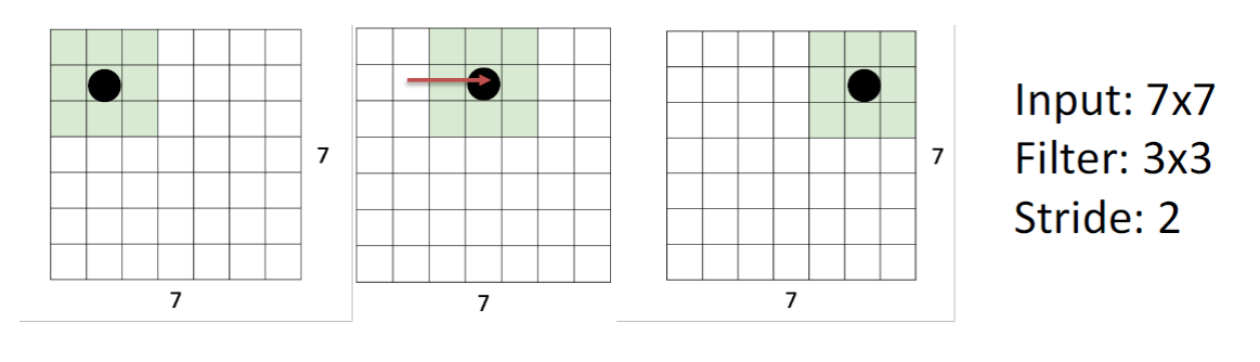


Stride

필터가 이미지를 훑으며 이동하는 간격

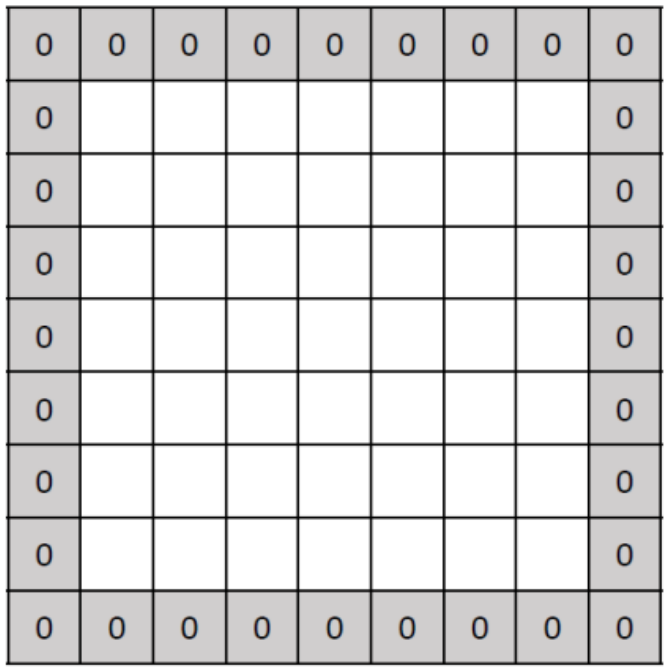
stride가 클수록 특징맵의 크기는 줄어들고 연산량이 줄어듦





Padding(패딩)

입력 데이터 외곽을 특정 값(0)으로 채워, 합성곱 연산 후에도 이미지 크기를 유지



Input: 7x7
Filter: 3x3
Output: 5x5

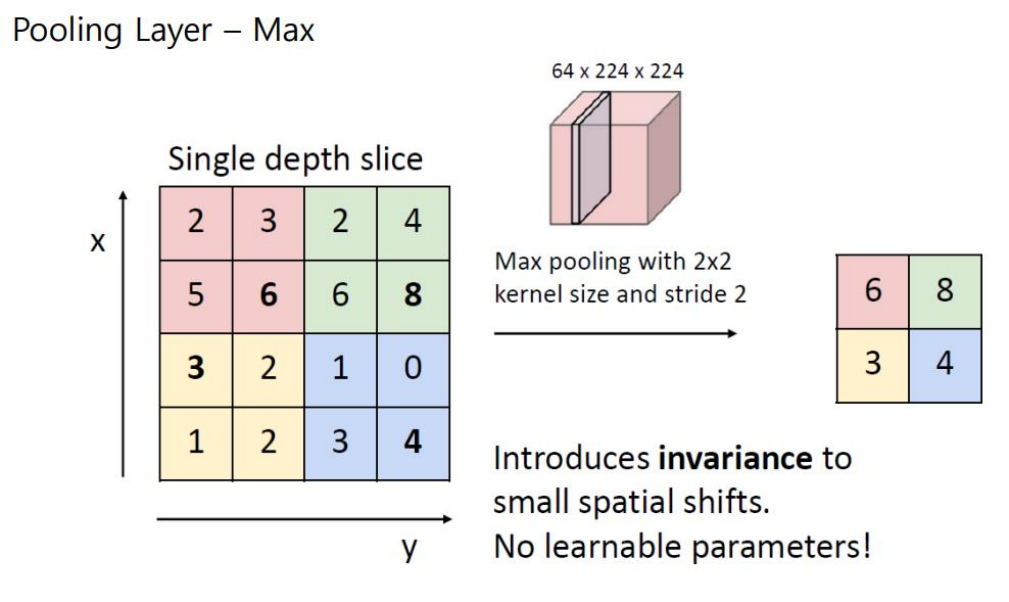
In general:
Input: W
Filter: K
Padding: P
Output: W – K + 1 + 2P

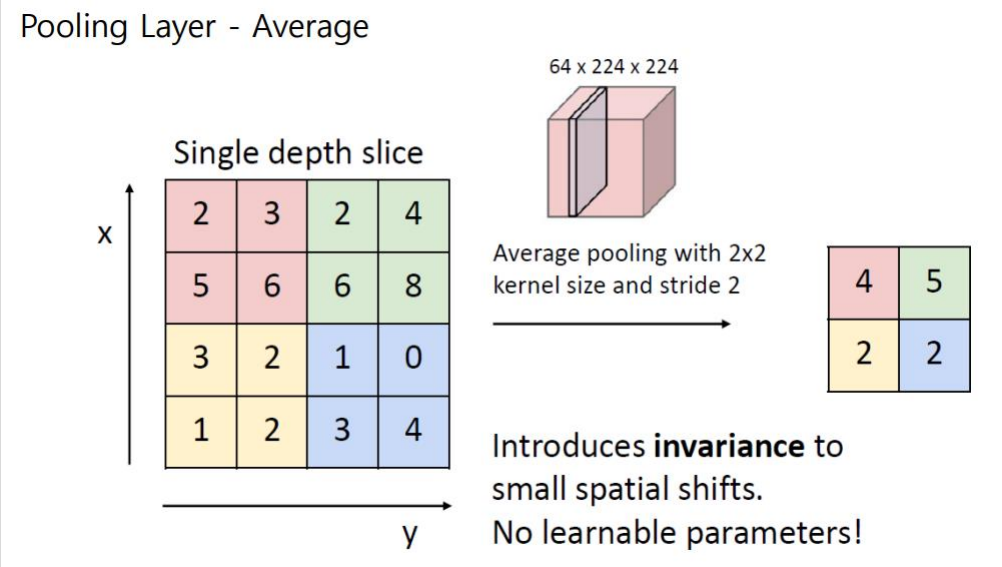
Very common: “same padding”
Set P = (K – 1) / 2
Then output size = input size

Pooling

특정 영역의 대표값
(Max,Average)을 추출

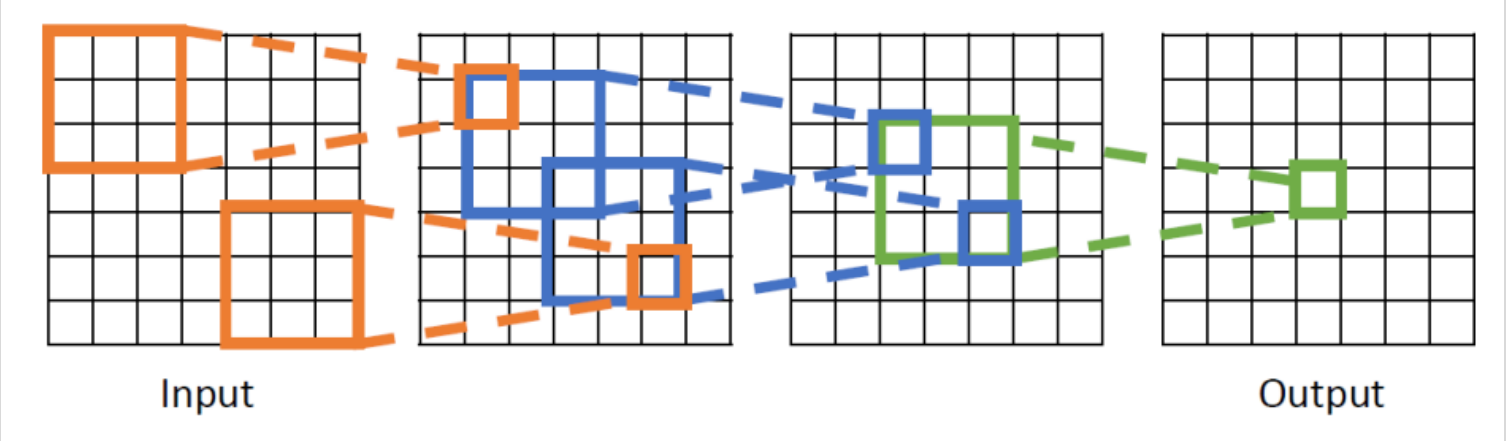
불변성(Invariance)을 높이고 과
적합을 방지



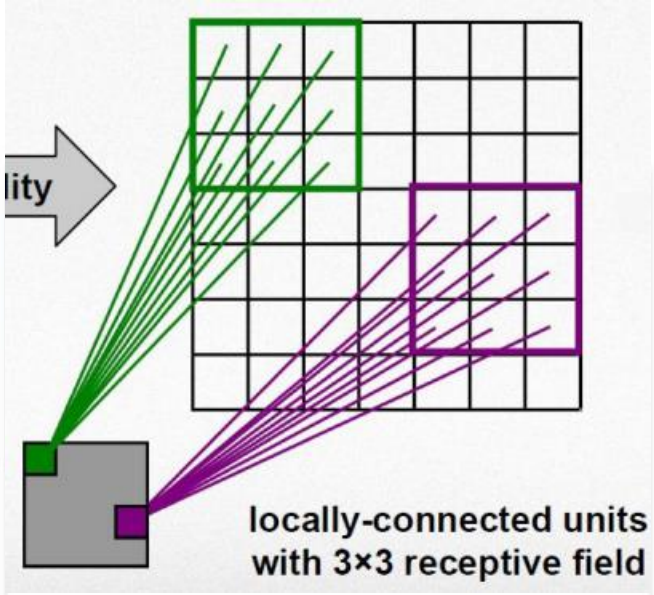


Receptive field(수용장)

receptive field size : 1 + L*(K-1)
L : layer
K : Kernel size

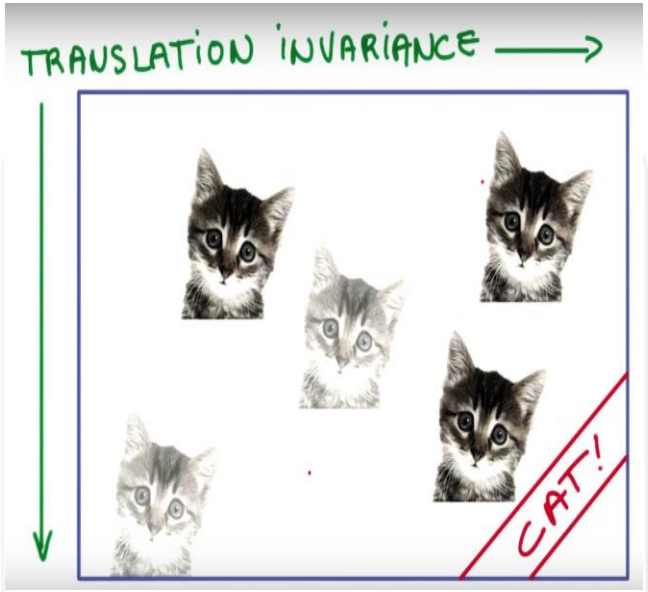


CNN의 특성



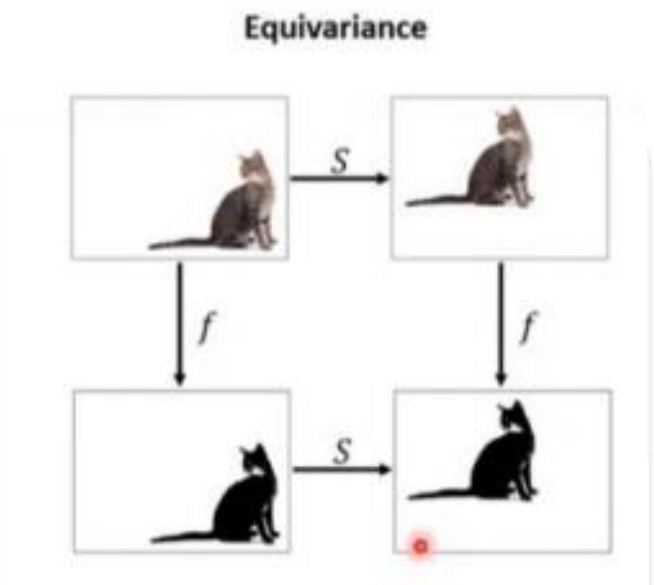
Locality (국소성)

CNN의 커널은 이미지 전체가 아닌 특정 영역 (Receptive Field)에 집중, 이를 통해 픽셀 간의 지역적 관계와 에지, 질감 같은 세밀한 패턴을 효과적으로 감지



Equivariance (등가성)

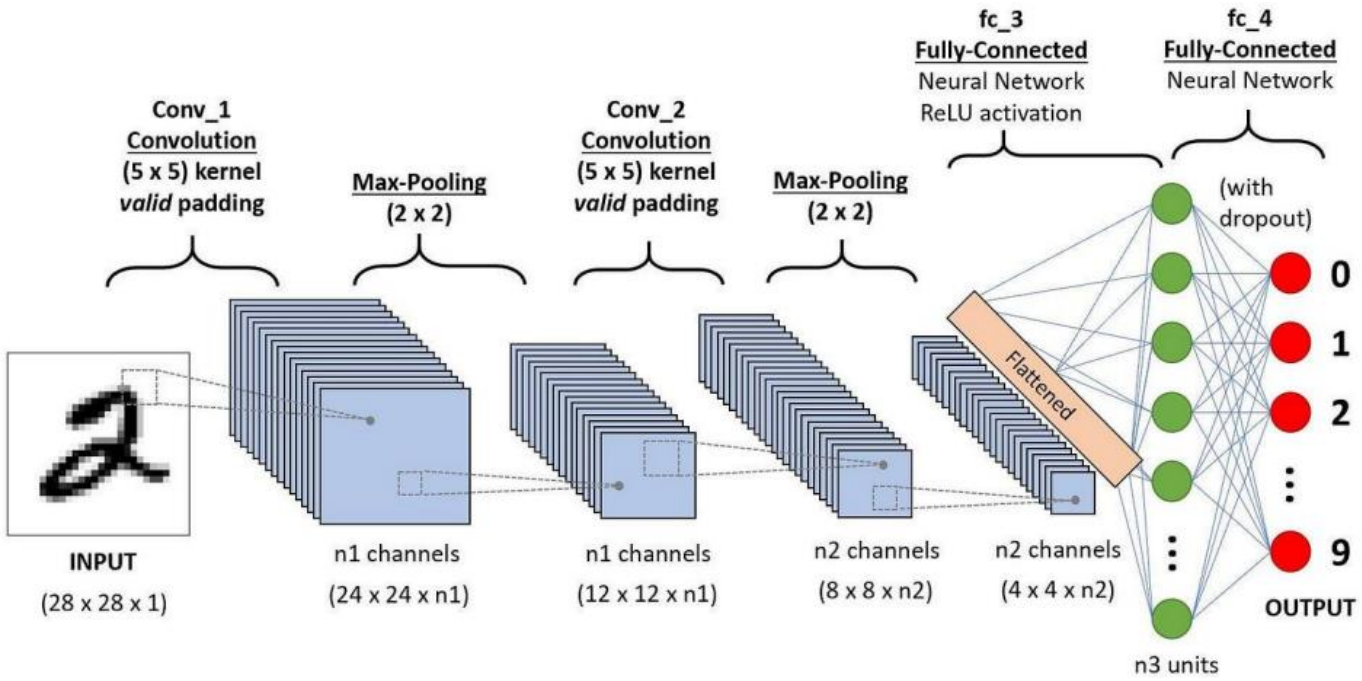
입력 데이터의 위치가 변하면 특징 맵(Feature Map)의 활성화 위치도 그에 맞춰 동일하게 이동



Invariance (불변성)

Pooling Layer와 깊은 계층 구조를 통해, 입력 객체가 조금 회전하거나 이동해도 최종 출력값은 크게 변하지 않음

Image Classification Pipeline



합성곱 층

특정 크기의 필터(Kernel)를 이미지 위에서 일정한 간격으로 이동시키며 연산을 수행

풀링 층

Pooling을 통해 이미지 크기를 줄여 연산량을 감소시키고, 위치 변화에 대한 불변성을 확보

Fully connected layer

핵심 특징들을 1차원으로 쭉 펼친 후, 마지막에 소프트맥스 함수를 사용하여 최종 클래스를 분류

일반화와 최적화 기법

$$\mathcal{L}(w) = \mathrm{MSE}(y, \hat{y}) + \sum_{i=0}^{N} \boxed{|w_i|}$$

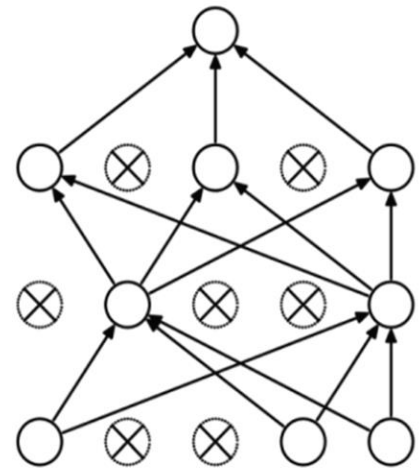
L1-Regularization (lasso)

$$\mathcal{L}(w) = \mathrm{MSE}(y, \hat{y}) + \sum_{i=0}^{N} \boxed{\|w_i\|_2^2}$$

L2-Regularization (ridge)

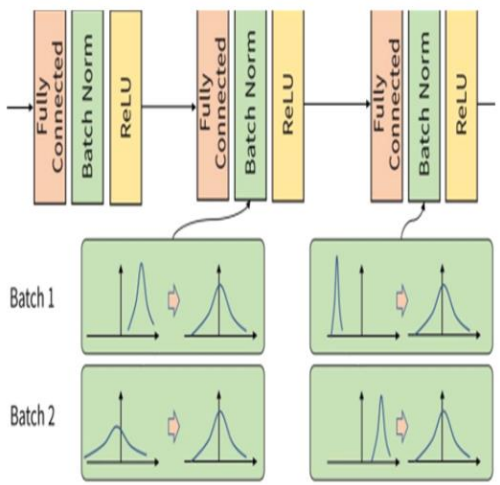
Regularization (정규화)

L1/L2 규제를 통해 가중치의 크기를 제한함으로써 모델의 복잡도를 낮춥니다. 이를 통해 특정 데이터에 과하게 적합되는 오버피팅을 방지합니다.



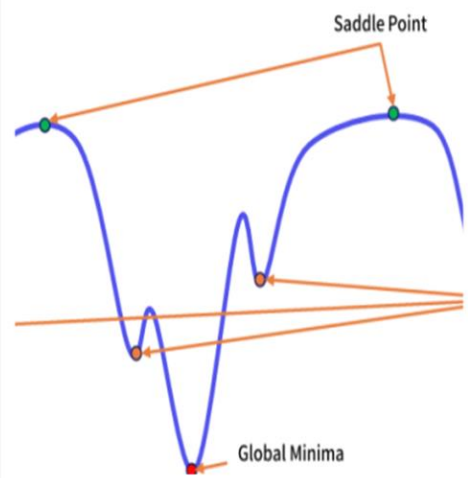
Dropout (드롭아웃)

훈련 과정에서 무작위로 일부 뉴런을 비활성화하여 학습시킵니다. 특정 뉴런에 대한 의존성을 줄입니다.



Batch Normalization

각 레이어의 입력을 정규화(평균 0, 분산 1)하여 학습 과정을 안정화합니다. 이를 통해 학습 속도가 획기적으로 개선되며 초기값 민감도를 낮출 수 있습니다.



Optimizers (최적화)

Momentum, RMSProp 등의 장점을 결합한 Adam과 같은 최적화 알고리즘을 사용하여, Local Minima 문제를 극복하고 빠르게 수렴합니다.

딥러닝이란?

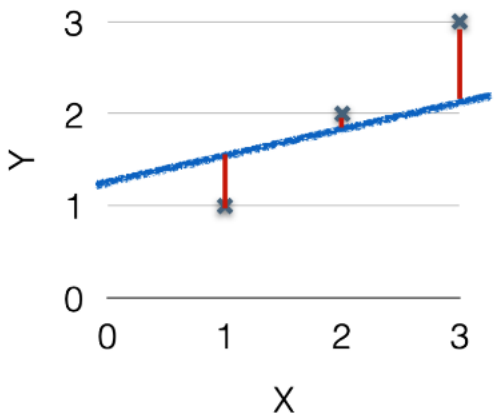
- 누가 질문하는가?
- 얼마나 많은 질문이 있는가?
- 어떻게 보정하는가?

선형회귀 이론

Regression

- Let's define a "cost function"
  - Cost function, Loss function, Objective function
  - Finding minimum values of cost function

$$H(x) = Wx + b$$

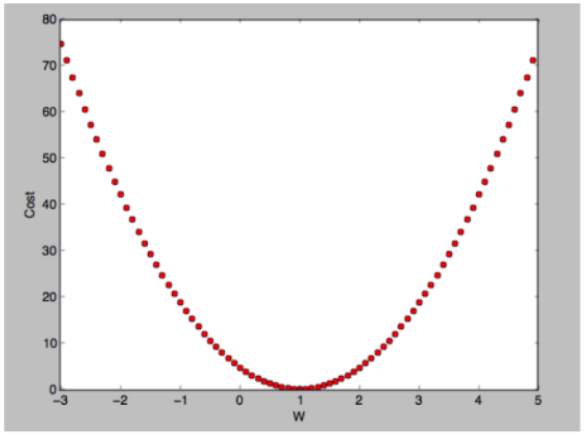


$$H(x) - y$$

$$H(x) - y \Rightarrow \frac{(H(x^{(1)}) - y^{(1)})^2 + (H(x^{(2)}) - y^{(2)})^2 + (H(x^{(3)}) - y^{(3)})^2}{3}$$

$$cost = \frac{1}{m}\sum_{i=1}^{m}(H(x^{(i)}) - y^{(i)})^2$$

$$cost(W) = \frac{1}{m}\sum_{i=1}^{m}(Wx^{(i)} - y^{(i)})^2$$

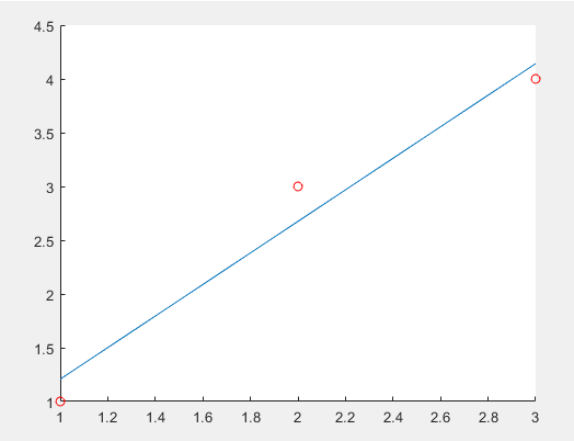


Gradient descent algorithm (경사하강알고리즘)

- Stochastic Gradient Descent : 확률적 경사하강 알고리즘

```python
# 데이터
x_train = torch.FloatTensor([[1], [2], [3]])
y_train = torch.FloatTensor([[1], [3], [4]])
# 모델 초기화
W = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
# optimizer 설정
optimizer = optim.SGD([W, b], lr=0.01)

nb_epochs = 1000
for epoch in range(nb_epochs + 1):

    # H(x) 계산
    hypothesis = x_train * W + b

    # cost 계산
    cost = torch.mean((hypothesis - y_train) ** 2)

    # cost로 H(x) 개선
    optimizer.zero_grad()
    cost.backward()
    optimizer.step()

    # 100번마다 로그 출력
    if epoch % 100 == 0:
        print('Epoch {:4d}/{} W: {:.3f}, b: {:.3f} Cost: {:.6f}'.format(
            epoch, nb_epochs, W.item(), b.item(), cost.item()
        ))
```

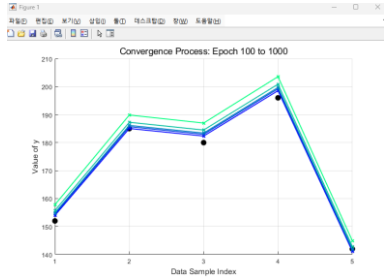
```
Epoch    0/1000 W: 0.127, b: 0.053 Cost: 8.666667
Epoch  100/1000 W: 1.213, b: 0.319 Cost: 0.116942
Epoch  200/1000 W: 1.274, b: 0.180 Cost: 0.093489
Epoch  300/1000 W: 1.323, b: 0.070 Cost: 0.078996
Epoch  400/1000 W: 1.361, b: -0.016 Cost: 0.070040
Epoch  500/1000 W: 1.390, b: -0.084 Cost: 0.064506
Epoch  600/1000 W: 1.414, b: -0.137 Cost: 0.061087
Epoch  700/1000 W: 1.432, b: -0.179 Cost: 0.058973
Epoch  800/1000 W: 1.447, b: -0.212 Cost: 0.057668
Epoch  900/1000 W: 1.458, b: -0.238 Cost: 0.056861
Epoch 1000/1000 W: 1.467, b: -0.259 Cost: 0.056362
```



다중선형회귀

$$H(x_1, x_2, x_3) = x_1 w_1 + x_2 w_2 + x_3 w_3 + b$$

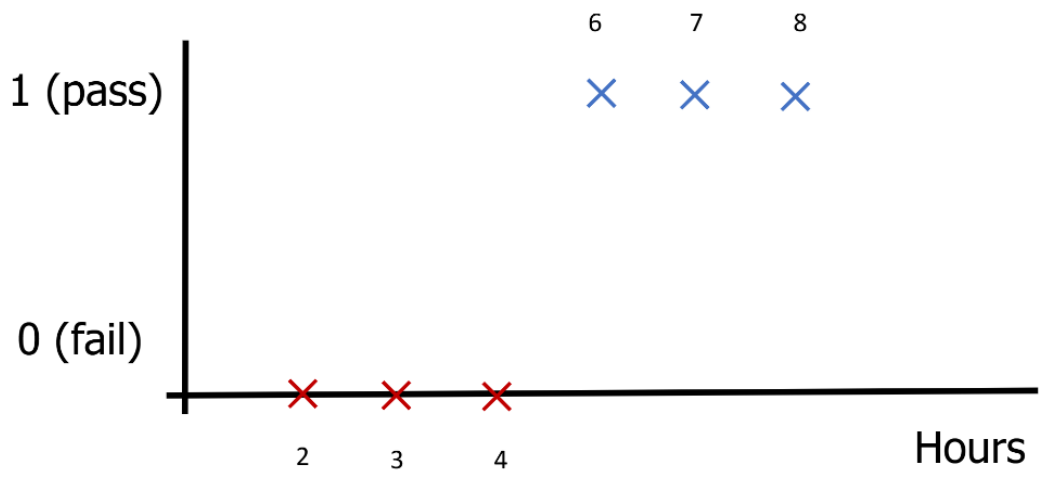
$$cost(W, b) = \frac{1}{m} \sum_{i=1}^{m} \left(H(x^{(i)}) - y^{(i)} \right)^2$$

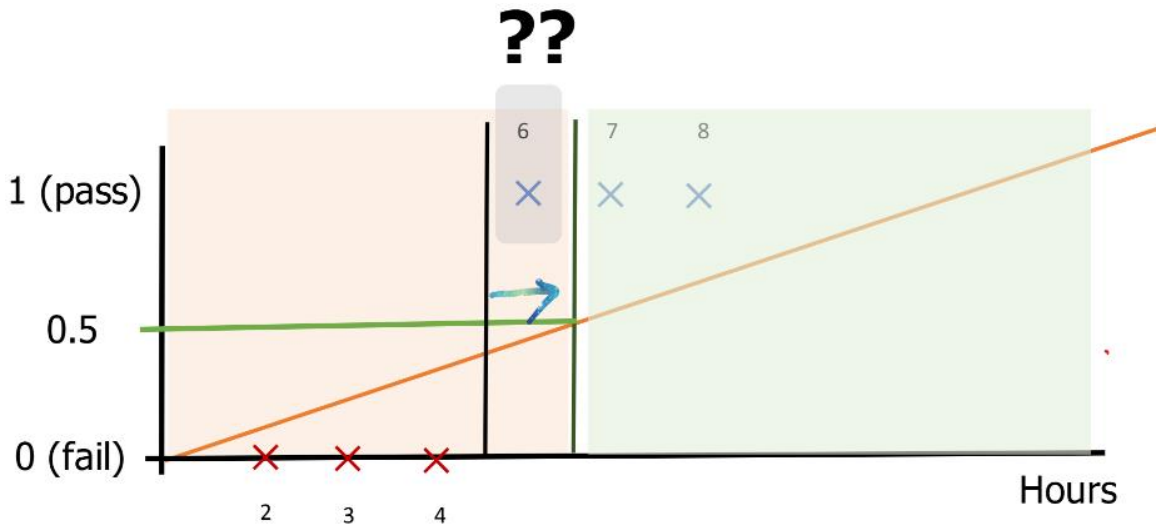


선형 분류(Linear Classification)

직선, 평면을 사용하여 서로 다른 클래스로 나누는 기법

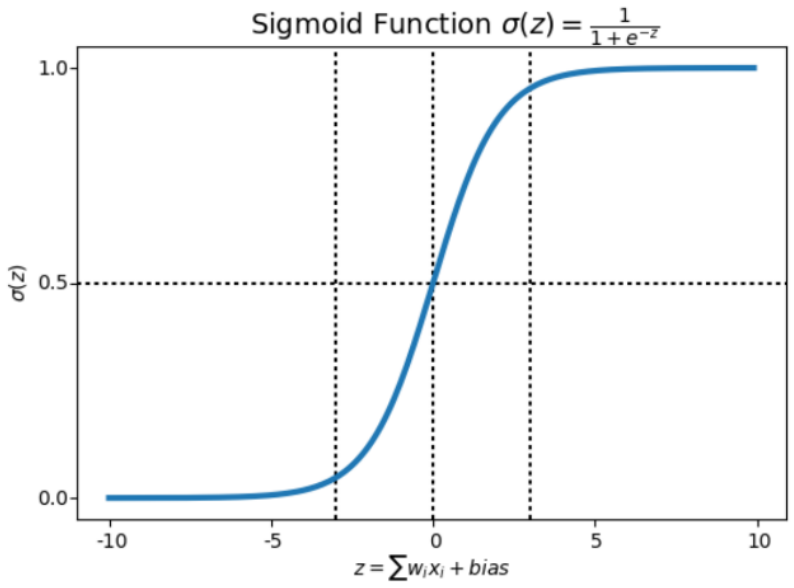
- Pass(1) / Fail(0)



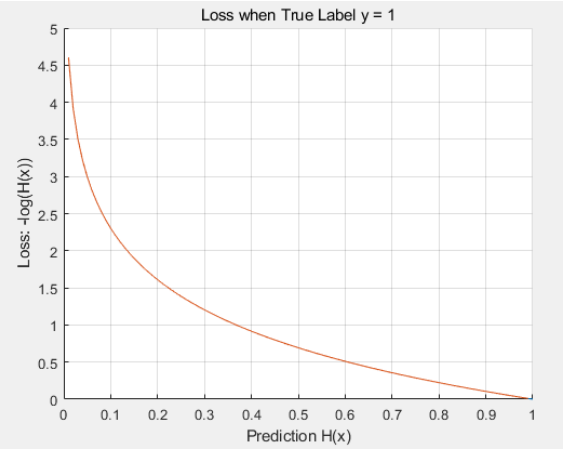


로지스틱 회귀(Logistic Regression)

특정 샘플이 특정 클래스에 속할 확률을 예측하는 모델



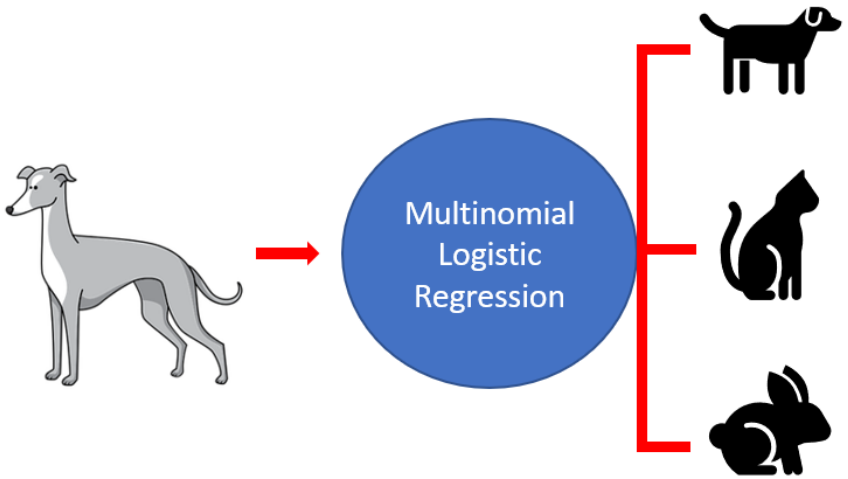
Z = WX

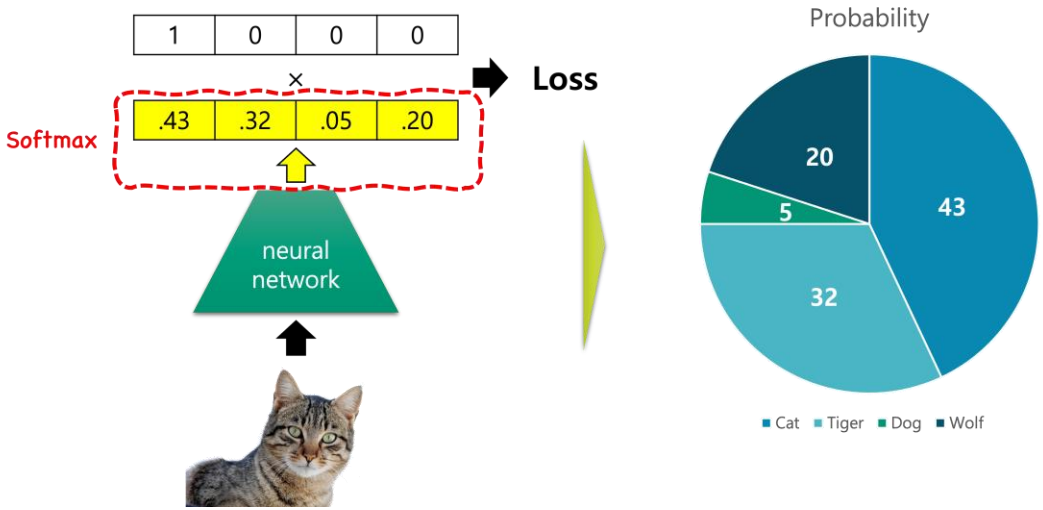


$$c(H(x), y) = \begin{cases} -log(H(x)) & : y = 1 \\ -log(1 - H(x)) & : y = 0 \end{cases}$$

Logistic function := sigmoid function

로지스틱 선형회귀 VS 소프트맥스 함수





Logistic regression

이진분류

클래스 개수 : 2개

Softmax

다중 분류

클래스 개수 : 3개 이상

출력값 여러 개의 합이 항상 1

- 누가 질문하는가? 데이터셋
- 얼마나 많은 질문이 있는가? 세상을 담을 수 있을 정도로 많이
- 어떻게 보정하는가? 경사 하강법을 통해 그 오차를 줄이는 방향으로 가중치를 조금씩 수정

딥러닝이란? 이러한 수많은 질문을 수렴시키는 가중치를 학습하는 것